Lazy Loading with Proxies

EF Core • Concise Lesson Summary

Goal: Configure EF Core to load related data automatically when a navigation property is accessed, without explicitly calling **Include()**.

1. Install the Proxies Package

Lazy loading through proxies requires **Microsoft.EntityFrameworkCore.Proxies**. Keep its major version compatible with the EF Core packages in your project.

```
dotnet add package Microsoft.EntityFrameworkCore.Proxies --version 8.0.0
```

2. Enable Lazy Loading

Add **UseLazyLoadingProxies()** to the DbContext options configuration.

```
var options = new DbContextOptionsBuilder<AppDbContext>()
    .UseLazyLoadingProxies()
    .UseSqlServer(connectionString)
    .LogTo(Console.WriteLine, LogLevel.Information)
    .Options;
```

LogTo() displays generated SQL in the console, helping you see queries triggered automatically at runtime.

3. Make Navigation Properties virtual

```
public virtual ICollection<Enrollment> Enrollments { get; set; }

// On Enrollment:
public virtual Student Student { get; set; }
public virtual Course Course { get; set; }
```

Why virtual? EF Core creates a proxy class derived from your entity. The proxy overrides virtual navigation properties and can intercept access to load related data.

4. What Happens at Runtime?

```
// First query: loads the student only
var student = context.Students
    .OrderBy(s => s.StudentId)
    .FirstOrDefault();

// Later access: can trigger another SQL query
var count = student.Enrollments.Count;
```

The first query loads the Student. When Enrollments is accessed, the proxy checks whether the navigation is loaded. If not, it asks EF Core to execute another query, provided the **DbContext is still available**.

5. SQL Behavior

```
-- First query
SELECT TOP(1) ...
FROM Students
ORDER BY StudentId;

-- Later, when Enrollments is accessed
SELECT ...
FROM Enrollments
WHERE StudentId = @StudentId;
```

The second query is triggered by property access, so it may not be obvious from reading the C# code alone.

6. The N+1 Problem

```
var students = context.Students.ToList();

foreach (var student in students)
{
    Console.WriteLine(student.Enrollments.Count);
}
```

1 Students query	50 Enrollment queries	51 Total queries

With 50 students, this example may run **1 query for Students + 50 queries for Enrollments = 51 queries**. This is the **N+1 Problem**, which can create many database round trips and hurt performance.

7. Requirements & Practical Notes

• **Package:** Microsoft.EntityFrameworkCore.Proxies, compatible with your EF Core version.

• **Configuration:** call UseLazyLoadingProxies().

• **Entities:** proxy entity classes must be inheritable (not sealed); navigation properties must be virtual.

• **Context lifetime:** the DbContext must still be available when unloaded navigation data is accessed.

• **Logging:** use LogTo() to inspect SQL and spot hidden queries.

• **Security:** EnableSensitiveDataLogging() can expose parameter values; use it only in controlled development environments.

Key Takeaways

Lazy Loading is convenient, but it can hide database queries. Monitor generated SQL and watch for N+1. For predictable backend/API queries, explicitly load only the related data the use case needs.

Mental model: Load the Student first → access the navigation property → proxy triggers a related-data query (if it is not already loaded and the context is available).